EcoHarmony Park

TP 6 · TDD · INSCRIBIRME A ACTIVIDAD

Decisiones de diseño

Grupo N° 2 · 4K1 · Ingeniería y Calidad de Software · Unidad 4: Aseguramiento de Calidad de Proceso y de Producto.

> COMO visitante QUIERO inscribirme a una actividad PARA reservar mi lugar en la misma.

Este documento explica cómo se diseñó la implementación de la User Story *Inscribirme a actividad* aplicando Desarrollo Conducido por Pruebas (TDD) y por qué se tomó cada decisión.

1. Alcance

La implementación cubre los criterios de aceptación y las pruebas de usuario de la User Story, más las reglas que definieron los Product Owners en el FAQ de la cátedra y en las respuestas a las consultas del grupo:

Regla	Origen
Actividades: Tirolesa, Safari, Palestra y Jardinería	User Story
El parque abre de martes a domingo de 8:30 a 19:00 y cierra el 25/12 y el 1/1	FAQ 1 y 11
Un turno por hora de 9 a 17; cada actividad dura una hora	FAQ 12, dominio
Cupos por turno: Safari 8, Palestra 12, Jardinería 12, Tirolesa 10	FAQ 13
Talle obligatorio en Tirolesa y Palestra, de XS a XXL	FAQ 9 y 14
Edad mínima: Tirolesa 8 años, Palestra 12; el resto sin límite	FAQ 15
La inscripción abre 24 horas antes del turno y cierra 15 minutos antes	FAQ 23 y 30
Una persona no puede estar en dos turnos del mismo horario	FAQ 29
Sin máximo de personas, salvo el cupo; si no alcanza se rechaza todo el grupo	FAQ 31, consulta 8
Términos y condiciones propios de cada actividad	User Story, FAQ 16
Correo de confirmación con el resumen y un QR de la inscripción	User Story, FAQ 28, consulta 7

Quedaron fuera del alcance, por indicación del PO, el registro y login (la sesión está simulada), la exigencia de tener una entrada comprada y el cobro, porque las actividades no tienen costo adicional.

2. Cómo se aplicó TDD

La funcionalidad se construyó de manera iterativa e incremental aplicando el ciclo **Red-Green-Refactor** a lo largo de 6 iteraciones sobre el dominio, los servicios, la infraestructura y la API:

- **Red:** en cada ciclo se escribieron primero las pruebas que expresan las nuevas reglas de negocio del incremento y se comprobó que fallaban.

- **Green:** a continuación se escribió el código productivo mínimo para hacer pasar esas pruebas, sin adelantar reglas que todavía no tenían prueba asociada.
- **Refactor:** con la suite de pruebas en verde se mejoró el diseño interno (extracción de puertos, desacoplamiento de repositorios y limpieza de validaciones), confirmando que el comportamiento no se rompía.

El historial del repositorio refleja esta evolución paso a paso en 6 ciclos (`test(red)` -> `feat(green)` -> `refactor`), separando además la incorporación posterior del frontend y de la documentación:

1. **Ciclo 1:** Reglas de apertura del calendario del parque y abstracción del reloj del sistema.
2. **Ciclo 2:** Modelos del dominio, configuración de actividades y generación de turnos iniciales.
3. **Ciclo 3:** Caso de uso principal de inscripción, validación de disponibilidad, parque abierto y descuento de cupos.
4. **Ciclo 4:** Reglas de negocio sobre visitantes (datos personales, talles, edades mínimas, ventana de 24 h a 15 min y superposición de horarios).
5. **Ciclo 5:** Generación de código QR en PNG y adaptadores de notificación por correo electrónico.
6. **Ciclo 6:** Exposición mediante endpoints HTTP en FastAPI y mapeo de excepciones a códigos de estado.

Las 94 pruebas automatizadas se reparten así:

Archivo	Pruebas	Qué verifica
`test_servicio_inscripcion.py`	51	Las reglas de negocio de la inscripción
`test_api.py`	16	Los endpoints, los códigos HTTP y los mensajes de error
`test_calendario_parque.py`	10	Días y horario de apertura, feriados
`test_datos_iniciales.py`	10	Horarios y cupos de cada actividad
`test_notificacion.py`	6	El contenido del correo, el QR y el envío por SMTP
`test_reloj.py`	1	La hora local del parque

Cada prueba de usuario de la User Story tiene al menos una prueba automatizada que la representa. Por ejemplo, "inscribirse a una actividad que no tiene cupo para el horario seleccionado (falla)" es `test_inscripcion_sin_cupo_en_el_horario_falla`, y la versión a través de la API es `test_post_inscripcion_sin_cupo_responde_409`. El README del repositorio tiene la tabla completa de trazabilidad.

Los nombres de las pruebas describen la regla y el resultado esperado, y cada prueba sigue la estructura Preparar, Actuar, Verificar. Así, la lista de pruebas también sirve como especificación ejecutable de la User Story.

3. Arquitectura del backend

```
backend/app/
├── dominio/          reglas de negocio, sin dependencias externas
├── servicios/        caso de uso "inscribir"
├── infraestructura/  implementaciones concretas: calendario, reloj,
│                     repositorios, QR y correo
├── api/              FastAPI: endpoints y esquemas de entrada y salida
└── main.py           arma las dependencias reales
```

Puertos e inyección de dependencias. El servicio de inscripción no conoce FastAPI, ni el servidor de correo, ni la hora del sistema. Recibe sus dependencias como interfaces (`typing.Protocol`):

- el repositorio de turnos y el de inscripciones,
- el calendario del parque y el reloj,
- el notificador de correo y el generador de QR.

Esta decisión surgió del propio TDD. Para escribir la prueba "parque cerrado" había que poder decidir desde la prueba si el parque estaba abierto, y para la ventana de 24 horas había que fijar la hora actual. Con dependencias inyectadas, las pruebas usan dobles (fakes y espías), corren en menos de un segundo y no dependen de una base de datos, de la red ni del día en que se ejecutan.

Reloj inyectado. La regla "desde 24 horas hasta 15 minutos antes" depende de la hora actual. Si el servicio llamara a `datetime.now()`, las pruebas pasarían o fallarían según el momento en que se ejecutan. Con el puerto `Reloj`, cada prueba fija el instante exacto que necesita: 24 horas antes, 15 minutos antes, el turno ya empezado. `RelojSistema` devuelve la hora de Córdoba, porque es la del parque, sin importar la zona horaria del servidor.

Capa de API fina. Los endpoints solo traducen HTTP a objetos del dominio y viceversa. La validación de formato (por ejemplo, el correo) la hace Pydantic, y las reglas de negocio las hace el servicio. Esto permite probar casi todas las reglas sin levantar un servidor.

4. Modelo de dominio

Clase	Responsabilidad
`Actividad`	Nombre, cupo por turno, si pide talle y edad mínima
`Turno`	Horario de inicio, cupos ocupados, ventana de inscripción y reserva de lugares
`Visitante`	Nombre, DNI, edad y talle, y la validación de esos datos
`SolicitudInscripcion`	Lo que pide el visitante antes de validarse
`Inscripcion`	La inscripción confirmada, con su código único y su correo

Experto en información. Durante los refactors cada regla se movió al objeto que tiene los datos para resolverla:

- La validación de nombre, DNI, edad y talle pasó a `Visitante.validar()`.
- El control de cupos pasó a `Turno.reservar()`.
- La ventana de inscripción pasó a `Turno.inscripcion_abierta()`.

- El talle obligatorio, el cupo y la edad mínima dejaron de ser listas sueltas de nombres y se convirtieron en atributos de `Actividad`.

Así, agregar una actividad nueva o cambiar su cupo es cambiar un dato del catálogo, sin tocar el servicio.

Una excepción por regla. Cada regla incumplida lanza su propia excepción, por ejemplo `SinCupoError`, `EdadInsuficienteError` o `InscripcionCerradaError`. Todas heredan de `InscripcionError`. La API las traduce a HTTP en un único manejador:

- 404 para una actividad o una inscripción inexistente,
- 409 para la falta de cupo o una inscripción superpuesta, porque son conflictos con el estado actual de los turnos,
- 422 para el resto.

Las pruebas del servicio verifican el tipo de error y no el texto, así que se pueden mejorar los mensajes sin romperlas.

Orden de validación. Primero se validan los datos que no requieren consultar nada (actividad, visitantes, talle, edad, términos). Después se consultan el calendario, el turno, la ventana de inscripción y las superposiciones. Recién al final se reservan los cupos, se registra la inscripción y se envía el correo. Hay pruebas que confirman que una solicitud rechazada no descuenta cupos ni envía correo.

5. Correo de confirmación y código QR

El PO pidió el QR real y aceptó el correo simulado en una ventana aparte que replique un cliente de correo.

- El backend genera un **QR real** en PNG con el código de inscripción, uno por inscripción, como respondió el PO.
- El backend arma el **correo completo** (asunto, cuerpo y QR adjunto). Ese correo se puede enviar por SMTP real configurando variables de entorno. Sin configuración, se guarda como archivo `.eml` en `backend/bandeja_salida/`.
- El endpoint `GET /inscripciones/{codigo}/correo` devuelve ese mismo correo, y el frontend lo muestra en una ventana con aspecto de cliente de correo. De este modo, lo que ve el visitante es el mensaje real que produce el sistema, no una copia escrita aparte en el frontend.

6. Frontend

El frontend es una aplicación web responsive hecha con React y Vite. Los PO aceptaron una aplicación web adaptada a celulares en lugar de una app nativa.

Flujo en pasos. El orden sigue el que describe el dominio: día, actividad, horario, participantes y, por último, el correo y los términos. Cada paso se habilita cuando el anterior está completo, y un resumen fijo muestra en todo momento qué falta para confirmar.

Guiar antes de enviar. El frontend repite las validaciones del backend para avisar a tiempo:

- deshabilita los lunes y los feriados (FAQ 2),
- muestra en cada turno los lugares que quedan y si la inscripción ya abrió o ya cerró,
- limita la cantidad de personas al cupo disponible,
- marca en cada participante los datos que faltan.

El backend sigue siendo la fuente de verdad: si una regla cambia o dos visitantes reservan a la vez, la respuesta del backend se muestra tal cual y los turnos se vuelven a consultar.

Regla del día. Sobre los horarios se muestra la franja de apertura del parque con la hora actual marcada. Así se entiende por qué los turnos de la mañana ya no están disponibles.

Términos por actividad. Los textos están en un único archivo de datos (`terminos.json`). La aplicación los muestra en un diálogo antes de aceptar, y el mismo archivo genera los PDF que se envían al PO para su validación. Así, el texto validado y el que ve el visitante son siempre el mismo.

Identidad visual. Se usan la paleta y la fuente Lato que indicó el PO (FAQ 3) y el logo del grupo. La ventana del correo usa tonos neutros a propósito, para que se reconozca como un cliente de correo y no como parte de la app del parque.

Accesibilidad.

- Las selecciones de día, actividad, horario y talle son grupos de opciones accesibles por teclado.
- Los errores de cada campo se asocian a su campo.
- El foco se ve en todos los controles.
- Se respetan las preferencias de movimiento reducido del sistema.

7. Estilos de código

- **Backend:** PEP 8, la guía de estilo definida por la comunidad de Python, verificada con flake8.
- **Frontend:** las Reglas de React definidas por la comunidad de React, verificadas con ESLint y el plugin oficial de hooks.

El detalle está en `ESTILOS_DE_CODIGO.md`.

8. Limitaciones conocidas

- Los turnos y las inscripciones se guardan en memoria y se pierden al reiniciar el backend. Para persistirlos alcanza con otras implementaciones de `RepositorioTurnos` y `RepositorioInscripciones`, sin cambiar el servicio ni las pruebas de dominio.
- El registro concurrente de dos inscripciones al mismo turno no está protegido con bloqueos, porque el almacenamiento en memoria corre en un único proceso. Con una base de datos haría falta una transacción en la reserva de cupos.
- La sesión del visitante está simulada; el correo se propone desde el usuario de prueba y se puede editar.